THE PRESENTATION · CS6650 Midterm Mastery Part 2 (empirical)

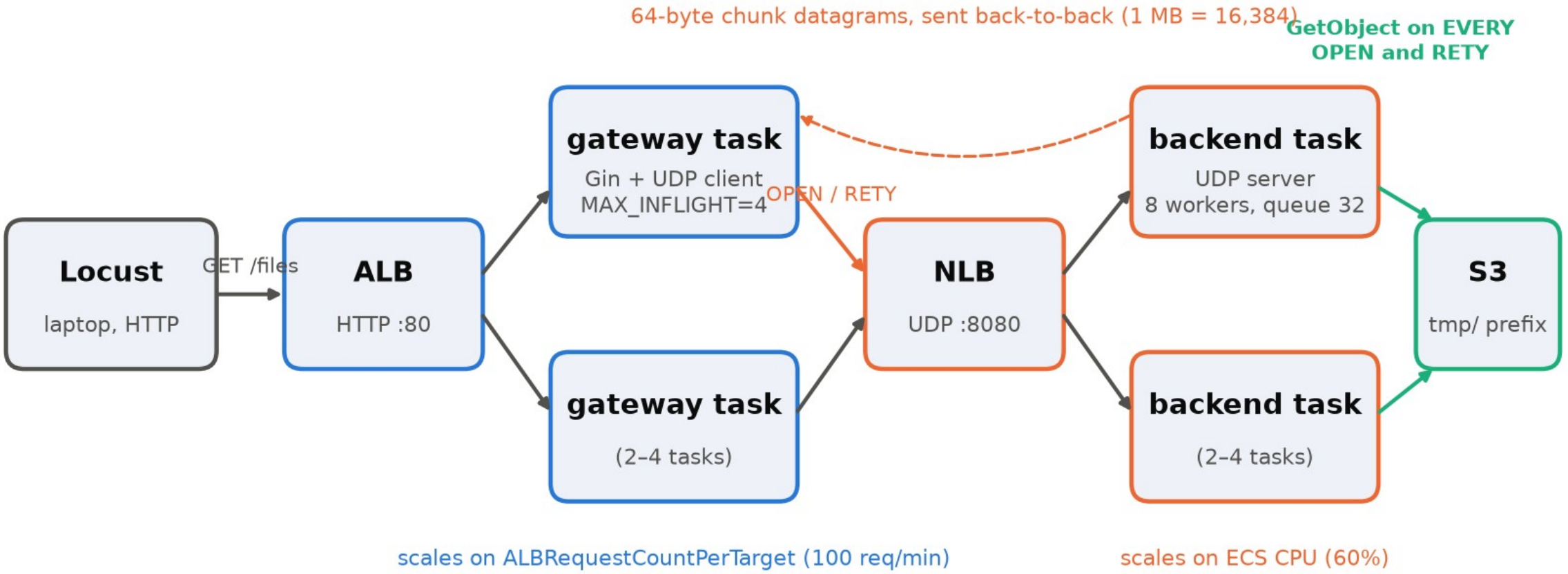
Stress-testing a UDP file-transfer service on AWS

Three experiments: loss × file size, retry budget, autoscaling under load

Nikhil · CS6650 Distributed Systems · Northeastern · November 2026

The service: HTTP in front, UDP behind

ALB → gateway (Gin + UDP client) → NLB → backend (UDP server) → S3



Two ECS Fargate services, 0.25 vCPU each, min 2 / max 4 tasks. Files are split into 64-byte datagrams.

Knobs, and the three experiments I picked

I added instrumentation: per-request X-* stats headers, CloudWatch EMF counters, and a per-request budget override

	Varied	Fixed	Measured
E1 loss × size	loss 0/1/5/10/20% × 4 KB/16 KB/64 KB/1 MB	budget 5 opens/5 rounds, 2 users	success, p50/p95/p99, RETYs, drops
E2 retry budget	rounds 1–20; opens 1–10	5% loss; 16 KB & 64 KB	success vs p95, RETYs per success
E3 scaling	step load 4→64 users	4 KB, 1% loss, default targets	task counts, 503s, latency, scale lag

3 × 60 s trials per cell for E1/E2 (1,843 requests); E3 is one 19-minute run (4,370 requests). Total cost ≈ $1–1.5 of Learner Lab credit.

Surprise: 0% simulated loss ≠ 0% loss

Chunks missing after the OPEN burst, with SEND_LOSS = RECV_LOSS = 0

35%

of a 1 MB file lost in the first burst → 0% success

11%

of a 64 KB file lost → 80% success

0%

of a 4 KB file lost → 100% success



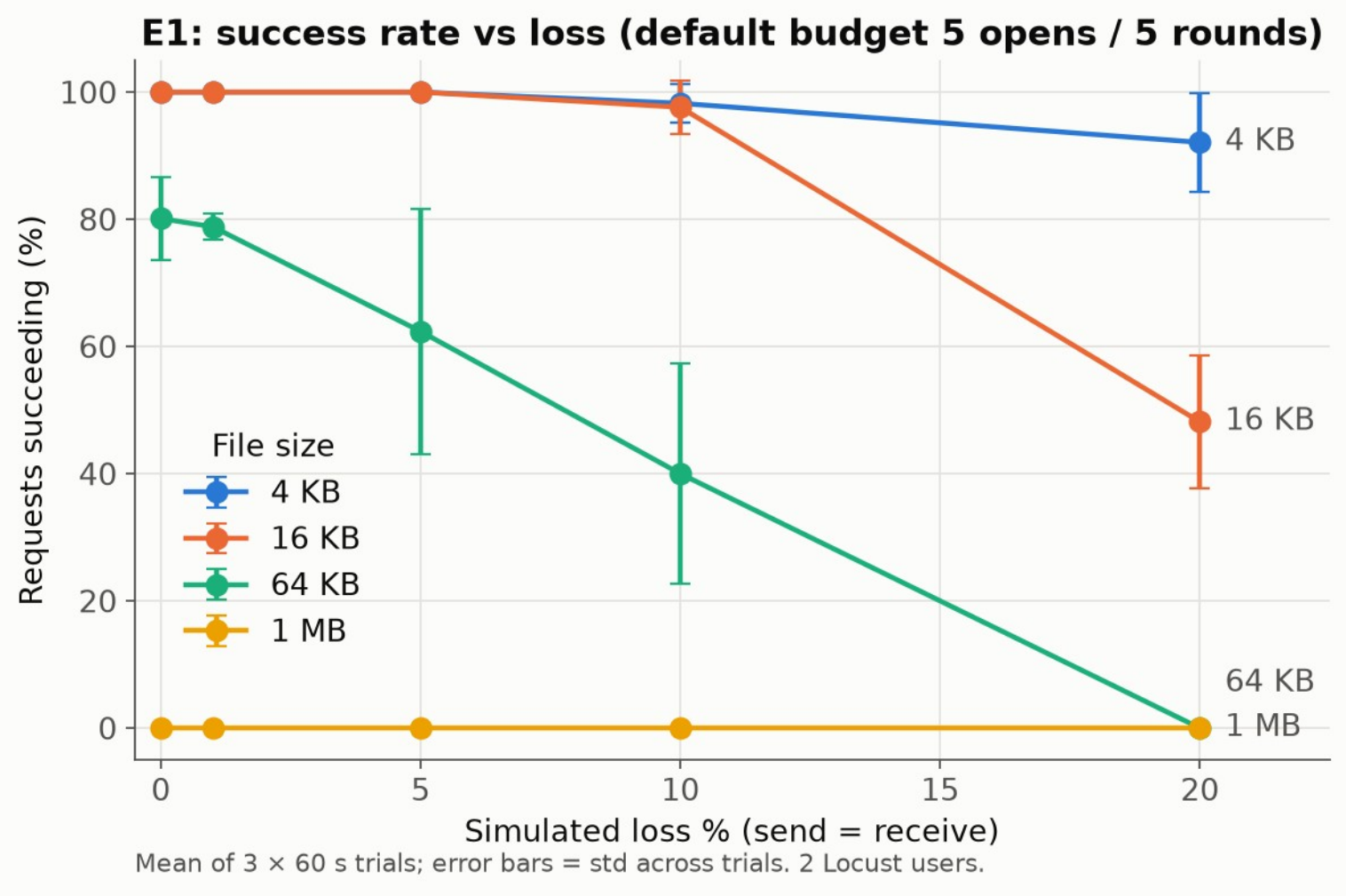
Not the NLB: bypassing it, 1 MB still lost 16–29%

Not gateway CPU: with a 1 vCPU gateway, 1 MB still lost 13–40%

Cause: the backend sends every chunk back-to-back with no pacing, so bursts overflow the receiver

E1: file size amplifies loss

Default budget (5 opens / 5 rounds), 2 Locust users, mean ± std of 3 trials



4 KB: ≥92% even at 20% loss

16 KB: cliff between 10% (98%) and 20% (48%)

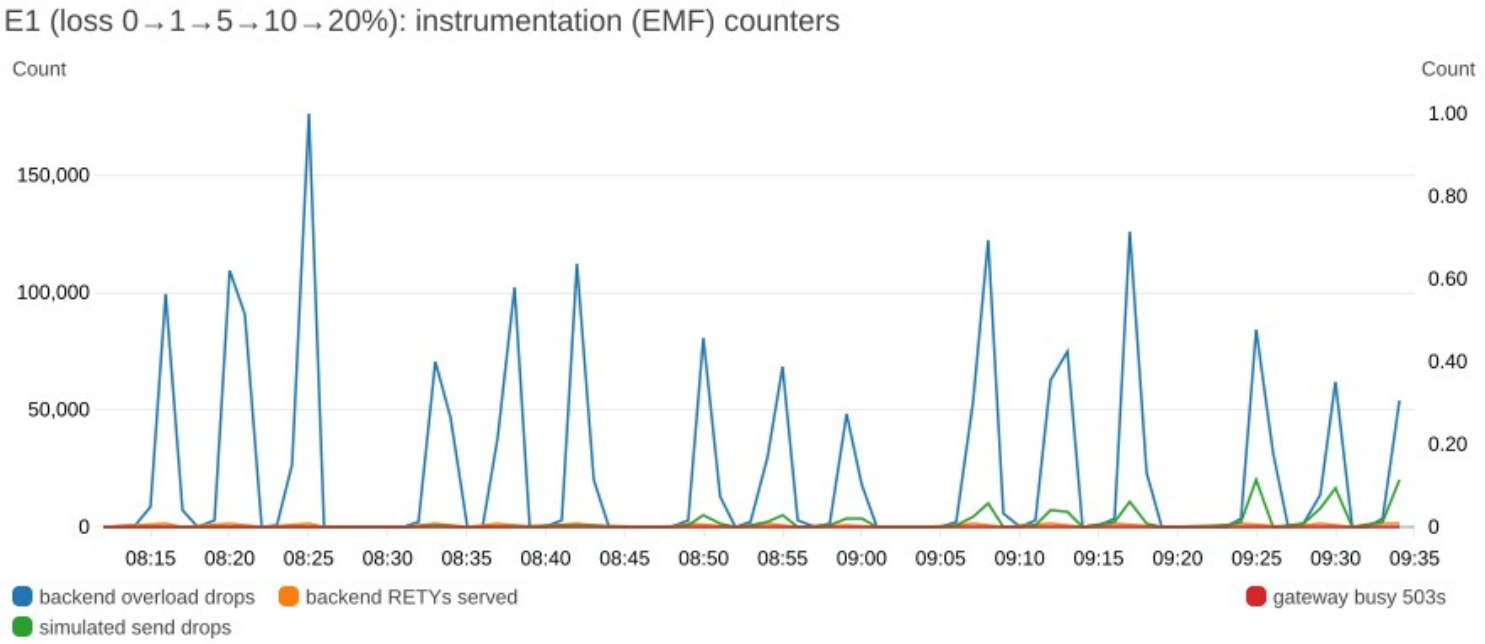
64 KB: 80% → 0% across the range

1 MB: 0% everywhere, about 19 s to fail

Latency rises in 2 s steps: p50 2.1 s → 10.2 s (4 KB, 0 → 20%)

E1: why retries can't catch up

Instrumentation (EMF) counters across E1; spikes = 1 MB runs



Each RETY = a full S3 GET: 31 ms (16 KB) → 155 ms (1 MB)

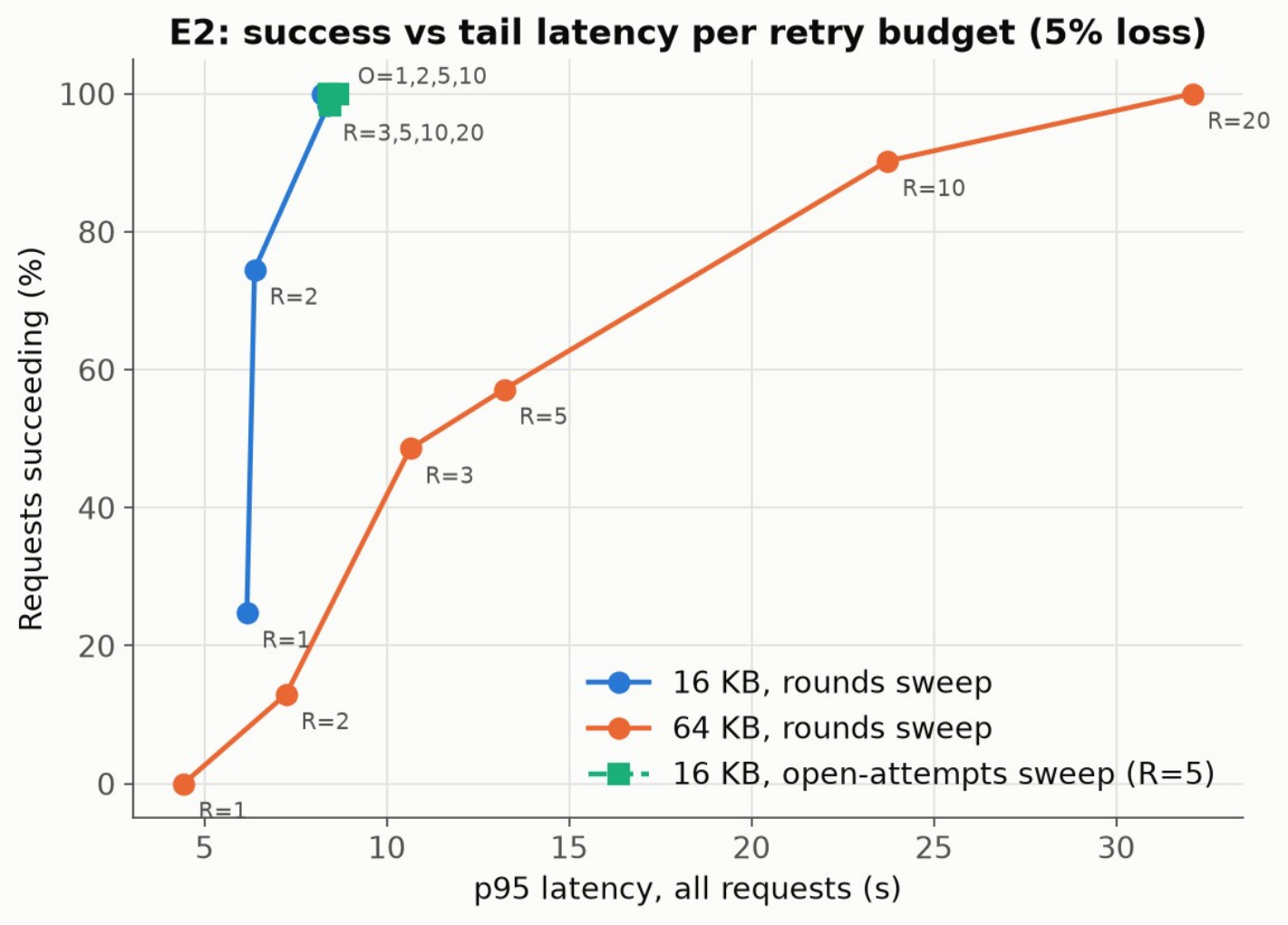
Queue = workers × 4 = 32: the rest are dropped silently

1 MB: 411k RETYs dropped vs 4.4k served in one cell

Orphaned work: the backend keeps serving RETYs after the client gave up

E2: rounds buy success, paid for in tail latency

5% loss; each point is one budget (R = retry rounds, O = open attempts)



16 KB: R = 3 gives 98%; the default 5 is fine

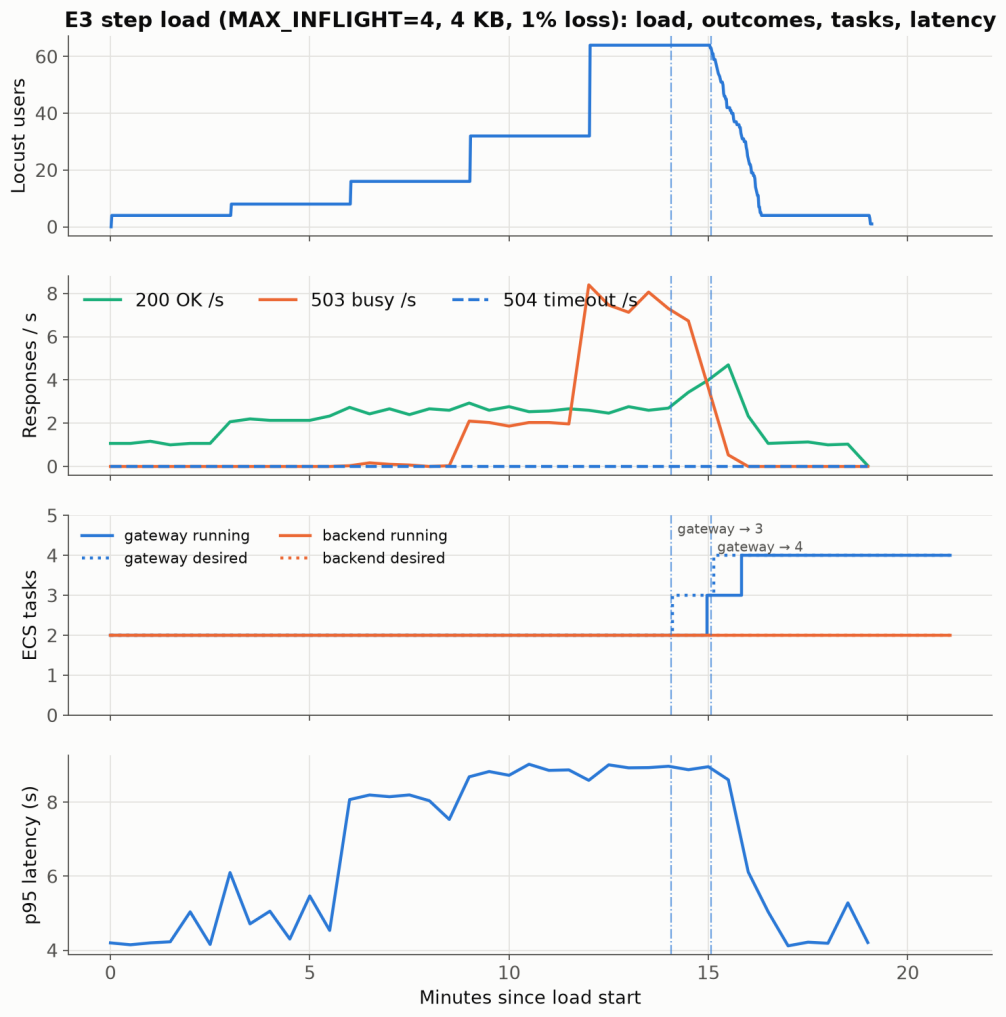
64 KB: default R = 5 gives 57%, R = 20 gives 100% but p95 32 s

Open attempts: O = 1 gave 98%, O = 2…10 gave 100% (≈1.05 OPENs per request)

R > ~25 would hit the ALB's 60 s idle timeout

E3: the gateway scales on its own rejections

Step load 4 → 64 users, 4 KB, 1% loss, stock thresholds; ECS polled every 10 s



Hard ceiling: 2 tasks × 4 slots ÷ ~3 s ≈ 2.7 OK/s, flat from 16 users on

Before scaling: 73% of requests are 503s at 64 users

Trigger: OK-only traffic peaked at 82 req/min/target < 100; 503s pushed it to 128+

Lag: ~5 min to the first scale-out, ~7 min to 4 tasks; 1,771 requests rejected meanwhile

E3: CPU scaling is blind here

Peak values during the 64-user step

2.9%

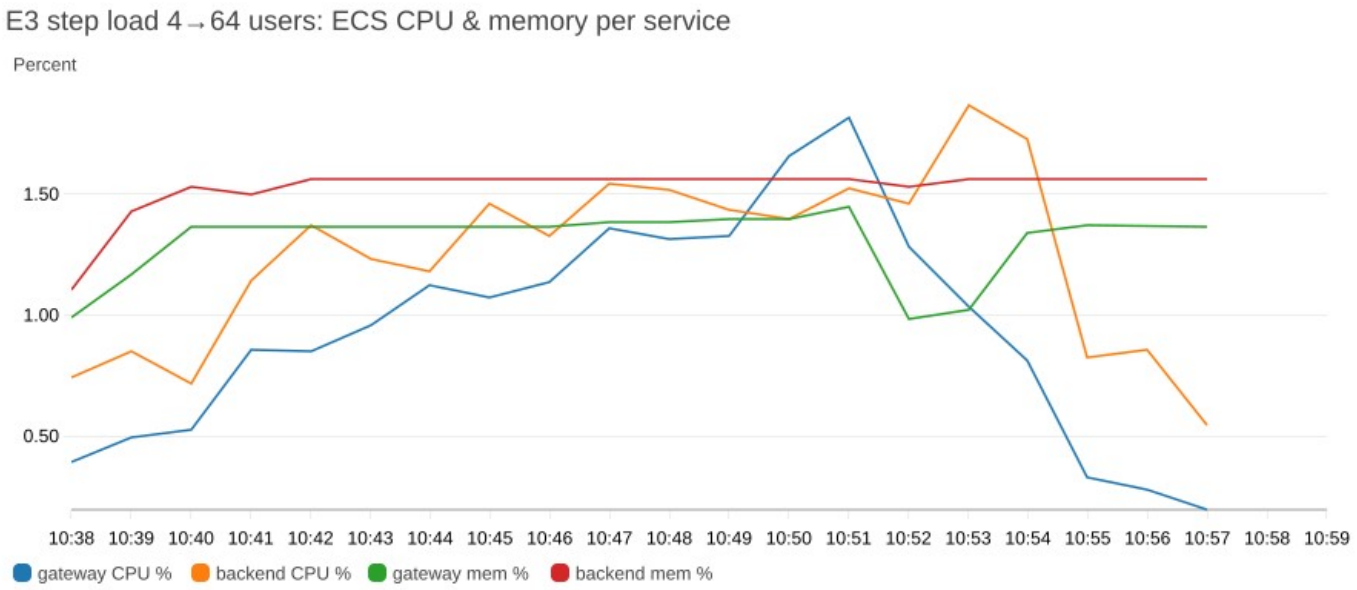
backend CPU max (target 60%), so the backend never scaled

7 / 8

backend workers busy at peak: the pool was nearly full

1.8%

gateway CPU max: CPU scaling would never fire here either



Both tiers are I/O- and timer-bound: 2 s quiet waits and S3 GETs, not computation

The NLB has no request metric, so the backend falls back to CPU, which never moves

Scale-in not seen in-window: AlarmLow needs 15 low minutes; the cooldown change can't shorten that

Zero 504s: under load the failure is admission (503), not transfer

Takeaways and recommendations

1

Pace the OPEN burst, or use bigger chunks

64-byte datagrams sent back-to-back lose 11–35% for files ≥ 64 KB before any simulated loss; 1 KB chunks would mean 16× fewer packets

2

Make RETY cheap: cache the file per transfer, batch the IDs

one S3 GET per RETY plus a 32-slot queue drops ~98% of retries; caching turns a 155 ms GET into a memory copy

3

Size-aware retry budget; keep open attempts low

R = 3 suffices ≤ 16 KB; 64 KB needs R ≈ 10–20; 1 open attempt was as good as 10

4

Scale on the real bottleneck, and don't let 503s drive it

gateway: in-flight utilisation or queue wait (EMF) instead of raw request count; backend: BusyWorkers / OverloadDrops instead of CPU; raise MAX_INFLIGHT, since tasks idle at under 2% CPU